WELT 2 · DOPPELSTUNDE 13 · LEVEL 2.1

# W2·1 – Die Rangliste: Standardbausteine und Suchen

Datei: `AQ_W2_L2_Rangliste_Geruest.txt` · IDE-Level 2.1: Basis · Erweiterung · Experte · Papier-Compiler: Aufgabe 3 · Im Lab: **Level 2.1**

Euer Dungeon-Spiel hat die ersten Testspieler. Jeder Lauf endet mit einem Eintrag in der **Highscore-Liste**: Name, Punkte und Spielzeit. Sofort kommen Fragen: Wer hat die meisten Punkte? Wie viele haben über 1000 geschafft? Steht „Dorn“ schon in der Liste? Hat sich jemand doppelt eingetragen?

Für all das gibt es **Standardbausteine** – kleine Algorithmen, die du im Abitur blind beherrschen musst. Sie stecken in fast jeder Implementierungsaufgabe: als ganze Methode oder als Teil einer größeren. Heute baust du sie für die Rangliste.

## Infoteil 1: Eintrag und Rangliste

Ein `Eintrag` ist eine Zeile der Highscore-Liste. Die `Rangliste` verwaltet ihre Einträge in einer **Reihung von Eintrag** – genau wie das Inventar des Helden in Welt 1. Beide Klassen sind im Gerüst schon angelegt, Eintrag sogar vollständig.

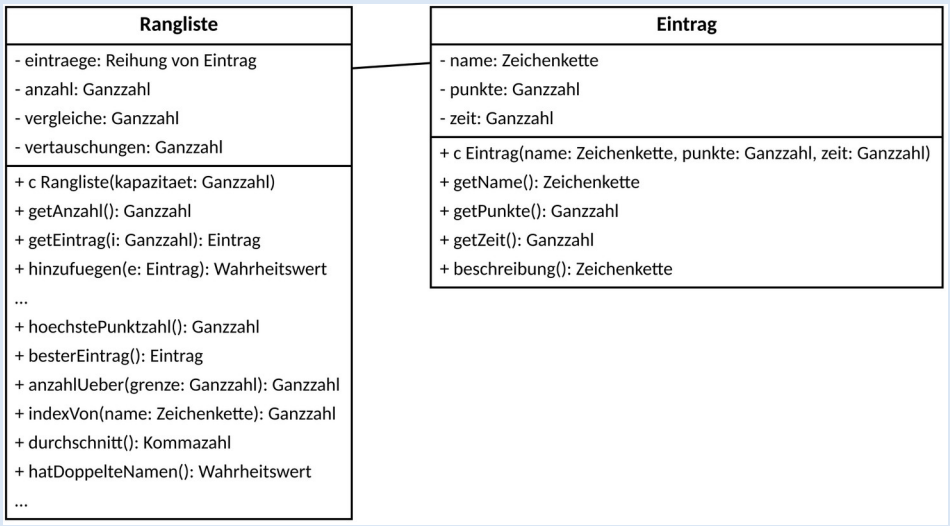



*Abbildung 1: Klassenkarten Rangliste und Eintrag (Ausschnitt bis Level 2.1)*

So sieht die Reihung der Demo-Rangliste aus dem Hauptprogramm aus (Kapazität 10):

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| **name** | Aria | Brann | Cyra | Dorn | Elin | Finn | **null** | **null** | **null** | **null** |
| **punkte** | 1200 | 800 | 1500 | 950 | 600 | 1100 | | | | |
| **zeit** | 95 s | 120 s | 80 s | 110 s | 140 s | 101 s | | | | |

- `anzahl` = 6: Belegt sind die Indizes **0 bis anzahl − 1**. Dahinter steht `null` – wer dort `getPunkte()` aufruft, erzeugt einen Laufzeitfehler.
- Alle Bausteine laufen deshalb `for (int i = 0; i < anzahl; i++)` – **nicht** bis `eintraege.length`.
- Die Standardbausteine **verändern die Liste nicht**. Sie lesen nur und liefern ein Ergebnis zurück.

## Aufgabe 0: Blitz-Warm-up: Zahlen, Plätze, null

PAPIER ★

Zum Start der Stunde: zehn Minuten, ohne Gerät. Jede Antwort ist kurz – aber jede steckt später in einem Baustein.

**a)** Gib den Wert jedes Ausdrucks an.

| Ausdruck | `7 / 2` | `(double) 7 / 2` | `(double) (7 / 2)` | `7 / 2.0` |
|---|---|---|---|---|
| **Wert** | | | | |

**b)** Eine Rangliste hat die Kapazität 10 und enthält 6 Einträge. Gib an, welche Indizes belegt sind, was in `eintraege[6]` steht und welcher Index der letzte gültige ist.

**c)** In `summe` steht die Summe aller Punkte (Ganzzahl). Schreibe **eine** Java-Zeile, die den Durchschnitt korrekt als Kommazahl in `schnitt` speichert.

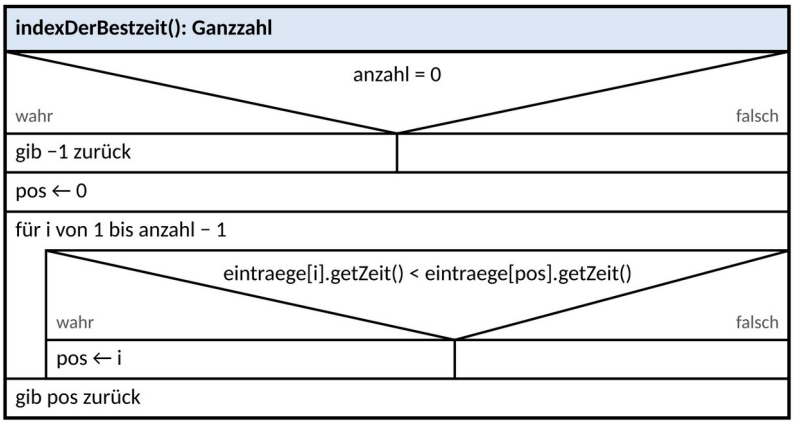
### Infoteil 2: Fünf Standardbausteine

Jeder Baustein folgt einem festen **Muster**. Wir zeigen jedes Muster an einer Beispielmethode, die **nicht** zum Projekt gehört (sie arbeitet mit der Spielzeit). In Level 2.1 überträgst du die Muster auf die Punkte und Namen. Jedes Muster siehst du in Java und als Struktogramm nach EH § 5.

## Baustein 1 – Minimum/Maximum mit Index

**Idee:** Der erste Eintrag ist der vorläufige Rekordhalter. Jeder weitere wird mit dem Rekord verglichen; ist er besser, wird er neuer Rekord. Wer sich den **Index** merkt, hat beides: Position und Eintrag. Hier: der Eintrag mit der kürzesten Spielzeit.

```
public int indexDerBestzeit() {
    if (anzahl == 0) {
        return -1;                      // leere Liste: kein Rekord
    }
    int pos = 0;                        // Rekordhalter: der erste
    for (int i = 1; i < anzahl; i++) {
        if (eintraege[i].getZeit() < eintraege[pos].getZeit()) {
            pos = i;                    // neuer Rekord
        }
    }
    return pos;
}
```

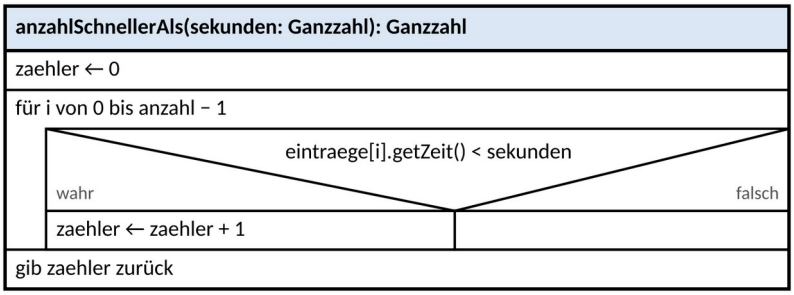



**Übertrage:** `hoechstePunktzahl()` und `besterEintrag()` – mit `>` statt `<`. Starte **nie** mit 0 als Rekord: Punkte können negativ sein (Strafpunkte). Mit `>` (nicht `>=`) bleibt bei Gleichstand der **erste**.

## Baustein 2 – Zählen

**Idee:** Ein Zähler startet bei 0 und wird für jeden passenden Eintrag um 1 erhöht. Hier: Wie viele Läufe waren schneller als `sekunden`?

```
public int anzahlSchnellerAls(int sekunden) {
    int zaehler = 0;
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getZeit() < sekunden) {
            zaehler++;
        }
    }
    return zaehler;
}
```

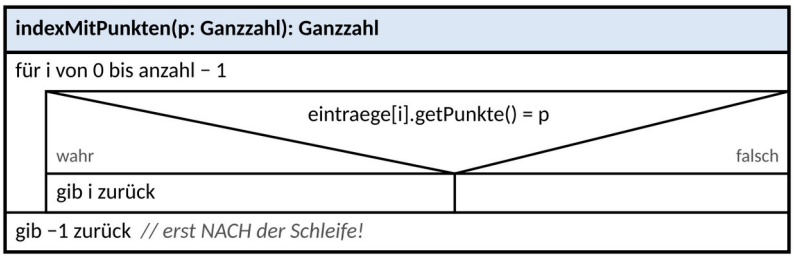


**Übertrage:** `anzahlUeber(grenze)` – gezählt werden Einträge mit **mehr** als `grenze` Punkten (`>`, nicht `>=`).

## Baustein 3 – Lineare Suche (erstes Vorkommen, sonst −1)

**Idee:** Von vorne nach hinten prüfen. Beim **ersten** Treffer sofort den Index zurückgeben – die Methode endet dort. Erst wenn die Schleife **ganz** durchgelaufen ist, steht fest: nicht vorhanden → −1.

```
public int indexMitPunkten(int p) {
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getPunkte() == p) {
            return i;                    // erster Treffer: fertig
        }
    }
    return -1;                           // erst NACH der Schleife!
}
```

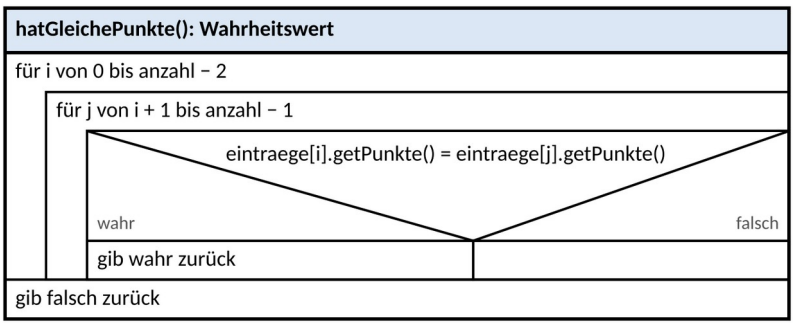



**Übertrage:** `indexVon(name)` – aber Namen sind Zeichenketten: Vergleich mit `equals`, nie mit `==`. Diese Methode schreibst du im **Papier-Compiler** (Aufgabe 3).

## Baustein 4 – Duplikate: jeder mit jedem

**Idee:** Zwei geschachtelte Schleifen bilden alle **Paare** (i, j) mit i < j. Die innere Schleife beginnt bei `i + 1`: So wird kein Eintrag mit sich selbst verglichen und kein Paar doppelt. Hier: Haben zwei Einträge dieselbe Punktzahl?

```
public boolean hatGleichePunkte() {
    for (int i = 0; i < anzahl - 1; i++) {
        for (int j = i + 1; j < anzahl; j++) {
            if (eintraege[i].getPunkte() == eintraege[j].getPunkte()) {
                return true;             // ein Paar genuegt
            }
        }
    }
    return false;
}
```

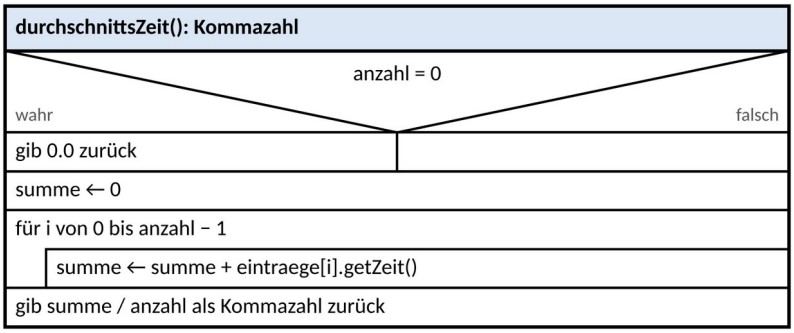



**Übertrage:** `hatDoppelteNamen()` – Namen mit `equals` vergleichen.

## Baustein 5 – Durchschnitt (Kommazahl-Division)

**Idee:** Erst summieren (Baustein Summe aus Welt 0), dann durch die Anzahl teilen. Zwei Fallen: Bei leerer Liste würdest du durch 0 teilen, und Ganzzahl / Ganzzahl rechnet **ganzzahlig**. Darum vor dem Teilen in eine Kommazahl umwandeln (Cast).

```
public double durchschnittsZeit() {
    if (anzahl == 0) {
        return 0.0;                      // nie durch 0 teilen
    }
    int summe = 0;
    for (int i = 0; i < anzahl; i++) {
        summe = summe + eintraege[i].getZeit();
    }
    return (double) summe / anzahl;      // Cast VOR der Division
}
```




**Übertrage:** `durchschnitt()` für die Punkte. Für die Demo-Liste liefert `durchschnittsZeit()` übrigens 107.666… – mit `summe / anzahl` wären es 107.

### Achtung Abi: Die fünf klassischen Baustein-Fehler

- **Rekord mit 0 starten:** falsch, sobald alle Werte negativ sind. Starte mit dem ersten Eintrag (vorher Leer-Abfrage!).
- **`==` bei Zeichenketten:** vergleicht in echtem Java die Referenzen, nicht den Inhalt. Immer `equals`. (Die Online-IDE verzeiht `==` – das Abitur nicht.)
- **Schleife bis `eintraege.length` oder `i <= anzahl`:** greift auf `null` oder hinter das Ende der Reihung zu.
- **`return -1` im else-Zweig der Suche:** die Suche bricht dann schon nach dem ersten Eintrag ab.
- **Ganzzahl-Division:** beim Durchschnitt und fehlende Leer-Abfrage (Division durch 0).

## Aufgabe 1: Das Rätsel der Bestzeit

PAPIER ★★

Die Spieleleitung vergibt einen Sonderpreis. Wer ihn bekommt, entscheidet diese Methode der Klasse Rangliste. Sie arbeitet auf der Demo-Rangliste aus Infoteil 1.

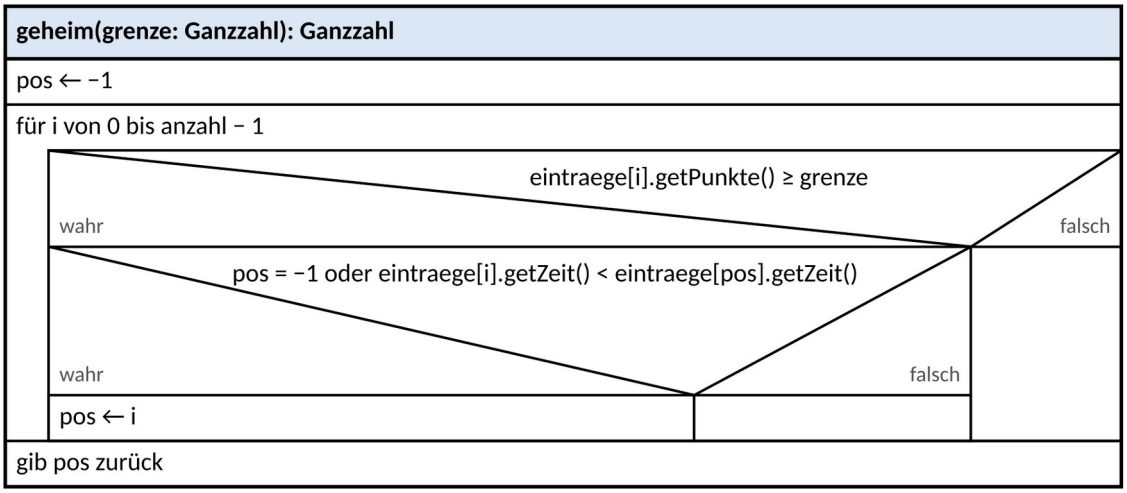



*Abbildung 2: Die Methode geheim*

**a)** Erstelle eine Tracetabelle für den Aufruf `geheim(1000)` auf der Demo-Rangliste und gib den Rückgabewert an. Trage eine Zeile vor der Schleife (Startwert von pos) und eine am Ende jedes Schleifendurchlaufs ein. **[3 BE]**

| i | Name | pos |
|---|---|---|
| – | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

Rückgabewert: _____

**b)** Gib die Rückgabewerte für `geheim(1200)` und `geheim(2000)` an. **[1 BE]**

**c)** Beschreibe, was die Methode allgemein berechnet, und schlage einen passenden Methodennamen vor. Nenne die beiden Standardbausteine, die hier kombiniert sind. **[3 BE]**

**d)** In Java wird die innere Bedingung zu `pos == -1 || eintraege[i].getZeit() < eintraege[pos].getZeit()`. Erläutere, warum die Reihenfolge der beiden Teilbedingungen hier entscheidend ist. **[2 BE]**

## Aufgabe 2: Die Rangliste füllt sich – Level 2.1, Teil 1 IDE ★

**a)** Lege einen Workspace **AlgoQuest_W2** an und darin eine Datei mit dem Inhalt von `AQ_W2_L2_Rangliste_Geruest.txt`. Lies die ANLEITUNG. Starte das Hauptprogramm einmal: Die Demo-Liste erscheint, die Bausteine liefern noch Platzhalterwerte.

**b)** Du arbeitest in der Klasse `Rangliste` unterhalb der Linie **AB HIER ARBEITEST DU**. Implementiere `hoechstePunktzahl()`, `besterEintrag()` und `anzahlUeber(int grenze)` nach den Mustern aus Infoteil 2.

**c)** **Stopp:** `indexVon` schreibst du zuerst auf Papier (Aufgabe 3). Lass die Methode bis dahin unverändert.

**d)** Starte im Testrunner die Testklasse `L21StandardTest`. Grün werden sollten jetzt die Tests zu deinen drei Methoden – darunter `experte_negativePunkte` und `experte_besterEintrag_gleichstand`. Die Stufen:

| Stufe | Tests in L21StandardTest beginnen mit … |
|---|---|
| ★ Basis | `basis_` |
| ★★ Erweiterung | `erweiterung_` |
| ★★★ Experte | `experte_` |

**Tipp:** Ein roter Test verrät in seiner Meldung den Fall, der nicht klappt – z. B. „starte nicht mit max = 0“. Lies sie, bevor du am Code drehst.

## Aufgabe 3: Der Papier-Compiler: indexVon

PAPIER → IDE ★★★

### Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (unten).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte `L21StandardTest`. Alle Tests mit **indexVon** im Namen grün beim ersten Lauf: **Papier-Compiler bestanden**. Sonst korrigierst du, bis alles grün ist.
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

**Auftrag:** Implementiere die Methode `indexVon(name: Zeichenkette): Ganzzahl` der Klasse Rangliste. Sie liefert den Index des **ersten** Eintrags mit genau diesem Namen. Kommt der Name nicht vor, liefert sie −1.

**a)** Implementiere die Methode in Java. **[4 BE]**

**b)** Abi-Check – hake ab, bevor du abtippst:

- ☐ Signatur wie gefordert, Rückgabe in **jedem** Pfad
- ☐ Schleife nur über die belegten Plätze (bis anzahl − 1)
- ☐ Namen mit `equals` verglichen, nicht mit `==`
- ☐ `return -1` steht **nach** der Schleife, nicht im else
- ☐ Leere Liste: liefert −1 ohne Laufzeitfehler

**c)** Ergebnis nach dem Abtippen: ☐ alle indexVon-Tests grün beim ersten Lauf ☐ grün nach Korrektur

### Fehlerlogbuch

| Nr. | Was war falsch? | Art: Syntax / Logik / Abi-Check | So vermeide ich es |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |

## Aufgabe 4: Jeder gegen jeden – Trace mit Doppelschleife

PAPIER ★★

Die Methode `hatGleichePunkte()` aus Baustein 4 wird auf einer Rangliste mit fünf Einträgen aufgerufen:

| Index | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| punkte | 700 | 900 | 650 | 1100 | 900 |

**a)** Erstelle eine Tracetabelle mit einer Zeile pro Vergleich – also bei jeder Auswertung der Bedingung eintraege[i].getPunkte() == eintraege[j].getPunkte() in Baustein 4 – und gib den Rückgabewert an. **[3 BE]**

| Vergleich Nr. | i | j | eintraege[i].getPunkte() | eintraege[j].getPunkte() |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

Rückgabewert: _____ Anzahl der Vergleiche: _____

**b)** Gib an, wie viele Vergleiche die Methode bei **5 Einträgen ohne gleiche Punkte** ausführt – und bei 6 Einträgen. Stelle eine Vermutung für n Einträge auf. **[3 BE]**

**c)** Begründe, warum die innere Schleife bei `i + 1` beginnt und nicht bei 0. **[2 BE]**

## Aufgabe 5: Bug-Hunt in der Rangliste

PAPIER ★★

Drei Teams haben ihre Bausteine abgegeben. In jeder Methode steckt **genau ein** Fehler. Alle Methoden stehen in der Klasse Rangliste.

**Methode A** – soll die höchste Punktzahl liefern (−1 bei leerer Liste):

```
1  public int hoechstePunktzahl() {
2      if (anzahl == 0) {
3          return -1;
4      }
5      int max = 0;
6      for (int i = 0; i < anzahl; i++) {
7          if (eintraege[i].getPunkte() > max) {
8              max = eintraege[i].getPunkte();
9          }
10     }
11     return max;
12 }
```

**Methode B** – soll den Index des ersten Eintrags mit diesem Namen liefern, sonst −1:

```
1  public int indexVon(String name) {
2      for (int i = 0; i < anzahl; i++) {
3          if (eintraege[i].getName() == name) {
4              return i;
5          }
6      }
7      return -1;
8  }
```

**Methode C** – soll zählen, wie viele Einträge mehr als `grenze` Punkte haben:

```
1  public int anzahlUeber(int grenze) {
2      int zaehler = 0;
3      for (int i = 0; i <= anzahl; i++) {
4          if (eintraege[i].getPunkte() > grenze) {
5              zaehler++;
6          }
7      }
8      return zaehler;
9  }
```

**a)** Gib für jede Methode die fehlerhafte Zeile an und korrigiere sie. **[3 BE]**

| Methode | Zeile | Korrektur |
|---|---|---|
| A | | |
| B | | |
| C | | |

**b)** Gib zu Methode A eine Rangliste an, für die sie ein falsches Ergebnis liefert. Nenne das falsche und das richtige Ergebnis. **[2 BE]**

**c)** Methode B besteht in der Online-IDE alle Tests mit festen Namen. Erläutere, warum sie in echtem Java trotzdem falsch ist und in welcher Situation das auffällt. **[2 BE]**

**d)** Beschreibe, was beim Aufruf von Methode C passiert – einmal für die Demo-Rangliste (Kapazität 10, 6 Einträge) und einmal für eine **volle** Rangliste. **[2 BE]**

## Aufgabe 6: Durchschnitt und Doppelgänger – Level 2.1, Teil 2

IDE ★ – ★★★

**a)** Implementiere `durchschnitt()` (Baustein 5, 0.0 bei leerer Liste) und `hatDoppelteNamen()` (Baustein 4, Vergleich mit `equals`).

**b)** Starte `L21StandardTest` und arbeite, bis **alle** Tests grün sind. Achte besonders auf `experte_volleListeUnveraendert`: Er prüft alle Bausteine auf einer **vollen** Rangliste.

**c)** **Vorhersage ohne Gerät:** Was gibt das Hauptprogramm für die Demo-Liste bei „Ueber 1000 Punkte“, „Index von Dorn“ und „Durchschnitt“ aus? Notiere, dann starte das Programm und vergleiche.

**Denkanstoß:** `hatDoppelteNamen` vergleicht jeden mit jedem. Wie viele Vergleiche wären das bei einer Rangliste mit 1000 Einträgen ohne doppelte Namen? Merke dir deine Zahl – in Welt 2·3 geht es genau um solche Rechnungen.

> 🎯 **Im Lab üben: Level 2.1 „Die Highscore-Liste – Standardbausteine und lineare Suche“ – Punkte gibt es dort automatisch**
>
> Das Lab-Level 2.1 im **Algo Quest Lab** (algoquest.frankjuchim.de) passt zu diesem Blatt. Die Aufgaben werden automatisch geprüft, Punkte gibt es beim ersten Lösen. Das IDE-Level 2.1 programmierst du weiterhin in der Online-IDE.

### Für Schnelle: Die Siegerehrung

Ergänze die Rangliste um eigene Methoden (sie haben keine Tests – prüfe sie im Hauptprogramm). Kombiniere dafür die Bausteine.

- `platzVon(name: Zeichenkette): Ganzzahl` – der Platz des ersten Eintrags mit diesem Namen in der **unsortierten** Liste: 1 + Anzahl der Einträge mit **mehr** Punkten. Kommt der Name nicht vor: −1. (Für die Demo-Liste: Dorn ist auf Platz 4.) Was passiert bei Punktgleichstand?
- `anzahlUeberDurchschnitt(): Ganzzahl` – wie viele Einträge liegen über dem Durchschnitt? Rufe dafür deine Methode `durchschnitt()` auf.
- `zweitbesterEintrag(): Eintrag` – mit **einem** Durchlauf durch die Liste (zwei Rekordhalter gleichzeitig). null, wenn es weniger als zwei Einträge gibt.

Side-Quest: freiwillig und ohne Punkte.